

Sistemas de Equações Lineares: Métodos Iterativos

Cálculo Numérico

Profa. Angela Leite Moreno

Universidade Federal de Alfenas

Conteúdo da Aula

- 1 Motivação: Por Que Métodos Iterativos?
- 2 Fundamentos dos Métodos Iterativos
- 3 Método de Gauss-Jacobi
- 4 Método de Gauss-Seidel
- 5 Convergência
- 6 Método SOR
- 7 Método do Gradiente Conjugado
- 8 Pré-Condicionamento
- 9 Comparação dos Métodos
- 10 Implementação Python
- 11 Exercícios
- 12 Resumo
- 13 Referências

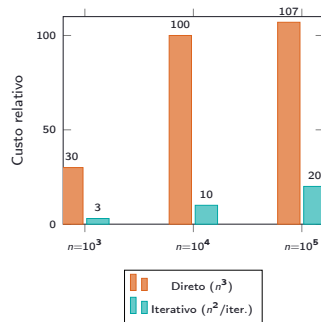
Quando os Métodos Diretos Têm Limites

Problemas com Gauss/LU em grande escala

- **Custo:** $\mathcal{O}(n^3)$ — inviável para $n \sim 10^6$.
- **Fill-in:** matrizes esparsas ficam densas durante a eliminação.
- **Memória:** armazenar $n \times n$ denso exige $\mathcal{O}(n^2)$.
- **Erros:** acúmulo de arredondamento para n enorme.

Métodos iterativos oferecem

- Custo **por iteração:** $\mathcal{O}(n^2)$ ou $\mathcal{O}(nnz)$.
- Exploração da **esparsidade**.
- Controle da **precisão** desejada.
- Facilidade de **paralelização**.



CFD em malha 100^3 : $n = 10^6$
Gauss: semanas. Iterativo: minutos!

O problema

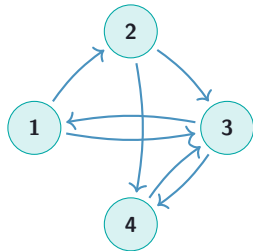
Classificar $n \sim 10^{11}$ páginas web resolvendo:

$$(I - \alpha P^T) \pi = \frac{(1 - \alpha)}{n} e$$

com $\alpha \approx 0,85$, P = matriz de transição esparsa.

Por que iterativo?

- Matriz P : $10^{11} \times 10^{11}$ — impossível armazenar!
- Cada página tem **10 links**: $\sim 0,000000001\%$ não nulos.
- Método das potências converge em **50–100 iterações**.



Cada nó = página web
Cada aresta = hiperlink

Ideia Central — Ponto Fixo Matricial

Transformação do sistema

O sistema $Ax = b$ é reescrito na forma de ponto fixo:

$$x = Cx + g,$$

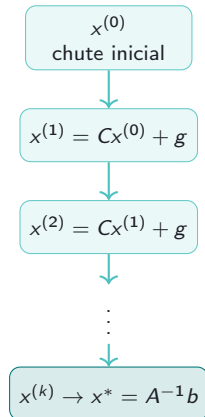
em que $C \in \mathbb{R}^{n \times n}$ é a **matriz de iteração** e $g \in \mathbb{R}^n$.

Iteração geral

Partindo de $x^{(0)}$ (chute inicial), computamos:

$$x^{(k+1)} = Cx^{(k)} + g, \quad k = 0, 1, 2, \dots$$

Se a sequência convergir: $\lim_{k \rightarrow \infty} x^{(k)} = \alpha = C\alpha + g$,
ou seja, α resolve $Ax = b$.



Critério de Parada

Quando parar?

Dada tolerância $\varepsilon > 0$, paramos quando o vetor estiver suficientemente próximo da iteração anterior. Medimos a distância entre $x^{(k)}$ e $x^{(k-1)}$ por:

$$d^{(k)} = \max_{1 \leq i \leq n} |x_i^{(k)} - x_i^{(k-1)}|.$$

Critério relativo (recomendado)

$$d_r^{(k)} = \frac{d^{(k)}}{\max_{1 \leq i \leq n} |x_i^{(k)}|} < \varepsilon.$$

O critério relativo evita parar cedo em soluções com componentes grandes.

Outros critérios práticos

- **Resíduo:** $\|Ax^{(k)} - b\| < \varepsilon$.
- **Norma da diferença:**
 $\|x^{(k)} - x^{(k-1)}\| < \varepsilon$.
- **Máximo de iterações:**
 $k > k_{\max}$ (segurança).

Atenção

$\|x^{(k)} - x^{(k-1)}\|$ pequeno **não garante** que $x^{(k)}$ esteja próximo de x^* — depende do condicionamento do sistema!

Decomposição $A = D - L - U$

Toda matriz A se decompõe como

$$A = D - L - U$$

- D : diagonal de A
- $-L$: triangular inferior estrita
- $-U$: triangular superior estrita

Convenção de sinais

Os sinais negativos em $-L$ e $-U$ são convenção para simplificar as fórmulas dos métodos.

Exemplo com $A 3 \times 3$

$$A = \begin{pmatrix} 4 & -1 & 0 \\ -2 & 5 & 1 \\ 0 & -1 & 3 \end{pmatrix}$$

$$D = \begin{pmatrix} 4 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 3 \end{pmatrix}$$

$$L = \begin{pmatrix} 0 & 0 & 0 \\ 2 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}, \quad U = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & -1 \\ 0 & 0 & 0 \end{pmatrix}$$

Verifique: $D - L - U = A \checkmark$

Derivação

De $Ax = b$ com $A = D - L - U$:

$$Dx = (L + U)x + b$$

$$x = \underbrace{D^{-1}(L + U)}_{C_J} x + \underbrace{D^{-1}b}_{g_J}$$

Iteração:

$$x^{(k+1)} = D^{-1}(L + U)x^{(k)} + D^{-1}b$$

Componente a componente

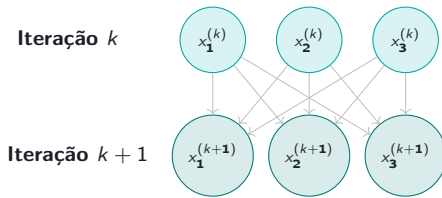
$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij} x_j^{(k)} \right)$$

Característica fundamental

Todos os $x_i^{(k+1)}$ são calculados utilizando **apenas** valores de $x^{(k)}$.

Vantagem: paralelizável — cada componente independente.

Custo por iteração: $\mathcal{O}(n^2)$



Gauss-Jacobi — Passo a Passo (Etapa 0)

Sistema de trabalho solução exata: $x^* = (1, -2, 1)^T$

$$\begin{cases} 10x_1 + 2x_2 + x_3 = 7 \\ x_1 + 5x_2 + x_3 = -8 \\ 2x_1 + 3x_2 + 10x_3 = 6 \end{cases}$$

Passo 1 — Verificar diagonal dominância:

$$\alpha_1 = \frac{|2|+|1|}{10} = 0,3 < 1 \checkmark$$

$$\alpha_2 = \frac{|1|+|1|}{5} = 0,4 < 1 \checkmark$$

$$\alpha_3 = \frac{|2|+|3|}{10} = 0,5 < 1 \checkmark$$

Passo 2 — Isolar cada variável:

$$x_1^{(k+1)} = \frac{1}{10} (7 - 2x_2^{(k)} - x_3^{(k)})$$

$$x_2^{(k+1)} = \frac{1}{5} (-8 - x_1^{(k)} - x_3^{(k)})$$

$$x_3^{(k+1)} = \frac{1}{10} (6 - 2x_1^{(k)} - 3x_2^{(k)})$$

Convergência garantida!

Ponto de partida

$$x^{(0)} = (0; 0; 0)^T$$

Gauss-Jacobi — Iteração $k = 1$

Regra de ouro: usar **somente** os valores de $x^{(0)}$ em todos os calculos

$$x^{(0)} = (0; 0; 0)^T$$

$$x_1^{(1)} = \frac{1}{10}(7 - 2 \cdot 0 - 0) = \frac{7}{10} = 0,700$$

$$x_2^{(1)} = \frac{1}{5}(-8 - 0 - 0) = \frac{-8}{5} = -1,600$$

$$x_3^{(1)} = \frac{1}{10}(6 - 2 \cdot 0 - 3 \cdot 0) = \frac{6}{10} = 0,600$$

Resultado de $k = 1$

$$x^{(1)} = \begin{pmatrix} 0,700 \\ -1,600 \\ 0,600 \end{pmatrix}$$

Critério de parada ($\varepsilon = 0,05$):

$$d_r^{(1)} = \frac{\max |x_i^{(1)} - x_i^{(0)}|}{\max |x_i^{(1)}|} = \frac{1,600}{1,600} = 1,00$$

1,00 > ε — continuar!

Gauss-Jacobi — Iteração $k = 2$

Usar somente $x^{(1)} = (0,700; -1,600; 0,600)^T$

$$\begin{aligned}x_1^{(2)} &= \frac{1}{10} (7 - 2 \cdot (-1,600) - 0,600) \\&= \frac{7+3,200-0,600}{10} = 0,960\end{aligned}$$

$$\begin{aligned}x_2^{(2)} &= \frac{1}{5} (-8 - 0,700 - 0,600) \\&= \frac{-9,300}{5} = -1,860\end{aligned}$$

$$\begin{aligned}x_3^{(2)} &= \frac{1}{10} (6 - 2 \cdot 0,700 - 3 \cdot (-1,600)) \\&= \frac{6-1,400+4,800}{10} = 0,940\end{aligned}$$

Resultado de $k = 2$

$$x^{(2)} = \begin{pmatrix} 0,960 \\ -1,860 \\ 0,940 \end{pmatrix}$$

$$d^{(2)} = \max \begin{Bmatrix} 0,260 \\ 0,260 \\ 0,340 \end{Bmatrix} = 0,340$$

$$d_r^{(2)} = \frac{0,340}{1,860} \approx 0,183$$

0,183 > 0,05 — continuar!

Tabela de iterações ($\varepsilon = 0,05$):

k	x_1	x_2	x_3	$d_r^{(k)}$
0	0,000	0,000	0,000	—
1	0,700	-1,600	0,600	1,000
2	0,960	-1,860	0,940	0,183
3	0,978	-1,980	0,988	0,064
4	0,997	-1,993	0,997	0,019
∞	1,000	-2,000	1,000	0

Convergiu em $k = 4$!

$$x^* \approx \begin{pmatrix} 0,997 \\ -1,993 \\ 0,997 \end{pmatrix}$$

Verificação

$$10(1) + 2(-2) + 1 = 7 \checkmark$$

$$1 + 5(-2) + 1 = -8 \checkmark$$

$$2 + 3(-2) + 10 = 6 \checkmark$$

Lembrete-chave

Em cada iteração, **todos** os $x_i^{(k+1)}$ usam exclusivamente os valores de $x^{(k)}$ — nenhum valor atualizado e aproveitado.

Gauss-Seidel — Use os Valores Recém-Calculados!

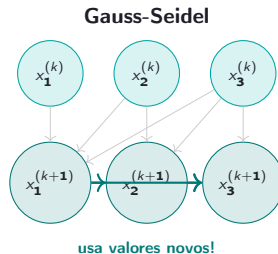
Ideia de melhoria

Jacobi usa apenas $x^{(k)}$ para calcular $x^{(k+1)}$. E se usarmos $x_1^{(k+1)}$ assim que o calcularmos?

Fórmula de Gauss-Seidel

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right)$$

Valores já atualizados são usados imediatamente!



Matriz de iteração

$$T_{GS} = (D - L)^{-1} U$$

Desvantagem

Não é paralelizável diretamente: cada $x_i^{(k+1)}$ depende dos anteriores.

Gauss-Seidel — Passo a Passo (Etapa 0)

Mesmo sistema — comparar com Jacobi!

$$\begin{cases} 10x_1 + 2x_2 + x_3 = 7 \\ x_1 + 5x_2 + x_3 = -8 \\ 2x_1 + 3x_2 + 10x_3 = 6 \end{cases} \quad x^{(0)} = (0; 0; 0)^T$$

Fórmulas de Gauss-Seidel:

$$x_1^{(k+1)} = \frac{1}{10} (7 - 2x_2^{(k)} - x_3^{(k)})$$

$$x_2^{(k+1)} = \frac{1}{5} (-8 - x_1^{(k+1)} - x_3^{(k)})$$

$$x_3^{(k+1)} = \frac{1}{10} (6 - 2x_1^{(k+1)} - 3x_2^{(k+1)})$$

Diferença crucial

- **Verde**: valor **já atualizado** nesta iteração
- **Azul**: valor da iteração **anterior**

Por que é melhor?

Assim que calculamos $x_1^{(k+1)}$, já o usamos para calcular $x_2^{(k+1)}$ — informação mais recente!

Gauss-Seidel — iteração $k = 1$

Partir de $x^{(0)} = (0; 0; 0)^T$

Calcular $x_1^{(1)}$ (só usa $x^{(0)}$):

$$x_1^{(1)} = \frac{1}{10}(7 - 2 \cdot 0 - 0) = 0,700$$

Calcular $x_2^{(1)}$ (já usa $x_1^{(1)}$!):

$$x_2^{(1)} = \frac{1}{5}(-8 - 0,700 - 0) = \frac{-8,700}{5} = -1,740$$

Calcular $x_3^{(1)}$ (usa $x_1^{(1)}$ e $x_2^{(1)}$!):

$$x_3^{(1)} = \frac{1}{10}(6 - 2 \cdot 0,700 - 3 \cdot (-1,740)) = \frac{6 - 1,4 + 5,22}{10} = 0,982$$

GS já está mais próximo de $(-2; 1)$ em x_2 e x_3 !

Resultado de $k = 1$

$$x^{(1)} = \begin{pmatrix} 0,700 \\ -1,740 \\ 0,982 \end{pmatrix}$$

Compare com Jacobi $k = 1$:

$$x_J^{(1)} = \begin{pmatrix} 0,700 \\ -1,600 \\ 0,600 \end{pmatrix}$$

Gauss-Seidel — iteração $k = 2$

Partir de $x^{(1)} = (0,700; -1,740; 0,982)^T$

Calcular $x_1^{(2)}$:

$$x_1^{(2)} = \frac{1}{10}(7 - 2 \cdot (-1,740) - 0,982) = \frac{7 + 3,480 - 0,982}{10} = 0,950$$

Calcular $x_2^{(2)}$ (usa $x_1^{(2)} = 0,950$):

$$x_2^{(2)} = \frac{1}{5}(-8 - 0,950 - 0,982) = \frac{-9,932}{5} = -1,986$$

Calcular $x_3^{(2)}$ (usa $x_1^{(2)}$ e $x_2^{(2)}$):

$$x_3^{(2)} = \frac{1}{10}(6 - 2 \cdot 0,950 - 3 \cdot (-1,986)) = \frac{6 - 1,900 + 5,958}{10} = 1,006$$

Resultado de $k = 2$

$$x^{(2)} = \begin{pmatrix} 0,950 \\ -1,986 \\ 1,006 \end{pmatrix}$$

$$d^{(2)} = 0,250; \quad d_r^{(2)} = 0,126$$

$0,126 > 0,05$ — mais uma!

Em $k = 3$:

$d_r^{(3)} \approx 0,028 < \varepsilon$ — parar!

$$x^{(3)} \approx (0,998; -1,999; 1,001)^T$$

Gauss-Seidel vs. Jacobi — Comparação Direta

Gauss-Seidel ($\varepsilon = 0,05$)

k	x_1	x_2	x_3	$d_r^{(k)}$
0	0,000	0,000	0,000	—
1	0,700	-1,740	0,982	1,000
2	0,950	-1,986	1,006	0,126
3	0,998	-1,999	1,001	0,028

Jacobi ($\varepsilon = 0,05$)

k	x_1	x_2	x_3	$d_r^{(k)}$
0	0,000	0,000	0,000	—
1	0,700	-1,600	0,600	1,000
2	0,960	-1,860	0,940	0,183
3	0,978	-1,980	0,988	0,064
4	0,997	-1,993	0,997	0,019

GS convergiu em 3 iterações
Jacobi precisou de 4

Ganho de $\sim 25\%$ neste exemplo.

Para sistemas maiores, o ganho cresce: GS tipicamente converge em **metade** das iterações de Jacobi.

Diferença essencial

- **Jacobi:** $x_2^{(1)} = -1,600$ (usa $x_1^{(0)} = 0$).
- **GS:** $x_2^{(1)} = -1,740$ (usa $x_1^{(1)} = 0,700$).

GS já usa informação **mais atual** na mesma iteração!

Jacobi vs. Gauss-Seidel — Comparação

Característica	Jacobi	Gauss-Seidel
Valores usados em $x_i^{(k+1)}$	Apenas $x^{(k)}$	$x^{(k+1)}$ (já calc.) + $x^{(k)}$
Memória necessária	2 vetores	1 vetor (in-place)
Paralelização	✓ Fácil	× Difícil
Velocidade de convergência	Linha de base	$\approx 2\times$ mais rápido
Matriz de iteração	$T_J = D^{-1}(L + U)$	$T_{GS} = (D - L)^{-1}U$

Quando Jacobi pode ser preferido

- Ambiente de **computação paralela**.
- Quando a **ordem de atualização** importa (alguns problemas físicos).

Quando GS é melhor

- Implementação **sequencial** (mais comum).
- Matrizes **SPD** (convergência garantida).
- Menor número total de iterações.

Convergência — Teorema Fundamental

Theorem (Condição Necessária e Suficiente)

A iteração $x^{(k+1)} = Cx^{(k)} + g$ converge para **qualquer** $x^{(0)}$ se, e somente se:

$$\rho(C) < 1,$$

em que $\rho(C) = \max_i |\lambda_i(C)|$ é o **raio espectral** de C .

Intuição: evolução do erro

Seja $e^{(k)} = x^{(k)} - x^*$. Da iteração:

$$e^{(k+1)} = C e^{(k)} \implies e^{(k)} = C^k e^{(0)}$$

Para $\rho(C) < 1$: $C^k \rightarrow 0$ e o erro desaparece!
Quanto menor $\rho(C)$, **mais rápida** a convergência.

Taxa de convergência

$$\|e^{(k)}\| \lesssim \rho(C)^k \|e^{(0)}\|$$

$\rho(C)$	iterações para 10^{-6}
0,9	~ 130
0,5	~ 20
0,1	~ 6

Critérios Práticos de Convergência

Critério das Linhas (Jacobi)

Se A é **estritamente diagonal dominante**:

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|, \quad \forall i$$

então Jacobi e Gauss-Seidel convergem para qualquer $x^{(0)}$.

$$\text{Define-se } \alpha_k = \frac{\sum_{j \neq k} |a_{kj}|}{|a_{kk}|}.$$

Se $\alpha = \max_k \alpha_k < 1$: converge!

Verificando o Exemplo

$$\begin{pmatrix} 10 & 2 & 1 \\ \dots & \dots & \dots \end{pmatrix}$$

Critério de Sassenfeld (Gauss-Seidel)

Defina $\beta_1 = \frac{\sum_{j>1} |a_{1j}|}{|a_{11}|}$ e, para $j > 1$:

$$\beta_j = \frac{\sum_{s<j} |a_{js}| \beta_s + \sum_{s>j} |a_{js}|}{|a_{jj}|}$$

Se $\beta = \max_j \beta_j < 1$: GS converge!
(Critério **mais forte** que o das linhas para GS.)

Se a matriz é SPD

- Gauss-Seidel **sempre** converge
- Jacobi converge se $2D - A$ for SPD

SOR — Successive Over-Relaxation

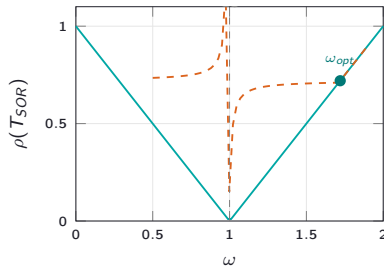
Ideia: acelerar Gauss-Seidel

Calcule o candidato Gauss-Seidel x_i^{GS} e faça uma **combinação convexa** com o valor atual:

$$x_i^{(k+1)} = (1 - \omega) x_i^{(k)} + \omega x_i^{GS}$$

O parâmetro ω

- $\omega = 1$: recupera Gauss-Seidel
- $0 < \omega < 1$: *under-relaxation* — mais estável
- $1 < \omega < 2$: *over-relaxation* — pode ser muito mais rápido!
- $\omega \geq 2$ ou $\omega \leq 0$: **diverge** sempre



Existe um ω_{opt} que **minimiza** $\rho(T_{SOR})$ e **acelera** a convergência!

SOR — ω Ótimo e Fórmula de Young

Theorem (Young, 1950)

Para certas classes de matrizes (incluindo tridiagonais e sistemas de EDPs), o parâmetro ótimo é:

$$\omega_{opt} = \frac{2}{1 + \sqrt{1 - \rho(T_J)^2}}.$$

Com ω_{opt} :

$$\rho(T_{SOR}) = \omega_{opt} - 1 < \rho(T_{GS}) < \rho(T_J).$$

Exemplo numérico

Se $\rho(T_J) = 0,9$:

$$\omega_{opt} = \frac{2}{1 + \sqrt{1 - 0,81}} \approx 1,72.$$
$$\rho(T_{SOR}) \approx 0,72 \text{ vs } \rho(T_J) = 0,9.$$

Ganho em EDPs 2D

Para equação de Poisson em malha $n \times n$:

Método	Iterações
Jacobi	$\mathcal{O}(n^2)$
Gauss-Seidel	$\mathcal{O}(n^2)$
SOR ótimo	$\mathcal{O}(n)$

Para $n = 100$: SOR é **100× mais rápido!**

Condição necessária

Para qualquer SOR convergir, é **obrigatório**:

$$0 < \omega < 2.$$

SOR — Passo a Passo (Etapa 0)

Mesmo sistema com $\omega = 1,2$ e $x^{(0)} = (0; 0; 0)^T$

$$\begin{cases} 10x_1 + 2x_2 + x_3 = 7 \\ x_1 + 5x_2 + x_3 = -8 \\ 2x_1 + 3x_2 + 10x_3 = 6 \end{cases}$$

Fórmula SOR em dois passos:

1. Calcule o candidato Gauss-Seidel:

$$x_i^{GS} = \frac{1}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_j^{(k+1)} - \sum_{j > i} a_{ij} x_j^{(k)} \right)$$

2. Aplique a relaxação:

$$x_i^{(k+1)} = \underbrace{(1 - \omega)}_{0,2} x_i^{(k)} + \underbrace{\omega}_{1,2} x_i^{GS}.$$

Intuição de $\omega > 1$

Com $\omega = 1,2$: damos um passo **20% maior** que o GS indicaria — “sobre-relaxamos” na direção correta.

$\omega = 1 \Rightarrow$ Gauss-Seidel puro

O SOR é uma **generalização** do GS.

SOR — iteração $k = 1$ (com $\omega = 1,2$)

Partir de $x^{(0)} = (0; 0; 0)^T$

Componente x_1 :

$$x_1^{GS} = \frac{1}{10}(7 - 2 \cdot 0 - 0) = 0,700$$

$$x_1^{(1)} = 0,2 \cdot 0 + 1,2 \cdot 0,700 = 0,840$$

Componente x_2 (usa $x_1^{(1)} = 0,840$):

$$x_2^{GS} = \frac{1}{5}(-8 - 0,840 - 0) = -1,768$$

$$x_2^{(1)} = 0,2 \cdot 0 + 1,2 \cdot (-1,768) = -2,122$$

Componente x_3 (usa $x_1^{(1)}$ e $x_2^{(1)}$):

$$x_3^{GS} = \frac{1}{10}(6 - 2 \cdot 0,840 - 3 \cdot (-2,122)) = \frac{10,686}{10} = 1,069$$

$$x_3^{(1)} = 0,2 \cdot 0 + 1,2 \cdot 1,069 = 1,282$$

Resultado de $k = 1$

$$x^{(1)} = \begin{pmatrix} 0,840 \\ -2,122 \\ 1,282 \end{pmatrix}$$

Compare com GS $k = 1$:

$$x_{GS}^{(1)} = \begin{pmatrix} 0,700 \\ -1,740 \\ 0,982 \end{pmatrix}$$

SOR com $\omega = 1,2$ “ultrapassou” um pouco — isso é intencional!

SOR — iteração $k = 2$ e Convergência

Partir de $x^{(1)} = (0,840; -2,122; 1,282)^T$

Componente x_1 :

$$x_1^{GS} = \frac{1}{10}(7 - 2 \cdot (-2,122) - 1,282) = \frac{9,962}{10} = 0,996$$

$$x_1^{(2)} = 0,2 \cdot 0,840 + 1,2 \cdot 0,996 = 1,363$$

Componente x_2 (usa $x_1^{(2)} = 1,363$):

$$x_2^{GS} = \frac{1}{5}(-8 - 1,363 - 1,282) = \frac{-10,645}{5} = -2,129$$

$$x_2^{(2)} = 0,2 \cdot (-2,122) + 1,2 \cdot (-2,129) = -2,979$$

Componente x_3 :

$$x_3^{GS} = \frac{1}{10}(6 - 2 \cdot 1,363 - 3 \cdot (-2,979)) = \frac{11,211}{10} = 1,121$$

$$x_3^{(2)} = 0,2 \cdot 1,282 + 1,2 \cdot 1,121 = 1,602$$

Resultado de $k = 2$

$$x^{(2)} = \begin{pmatrix} 1,363 \\ -2,979 \\ 1,602 \end{pmatrix}$$

Atenção!

$\omega = 1,2$ causou **oscilações** em torno da solução. O método ainda converge, mas com ω próximo do ótimo a convergência é mais suave.

Tabela resumida

k	x_1	x_2	x_3
0	0,000	0,000	0,000
1	0,840	-2,122	1,282
2	1,363	-2,979	1,602
...	...	...	...
x^*	1,000	-2,000	1,000

Gradiente Conjugado — A Grande Ideia

Para matrizes SPD

Resolver $Ax = b$ (A SPD) é equivalente a minimizar a função energia:

$$f(x) = \frac{1}{2}x^T Ax - b^T x$$

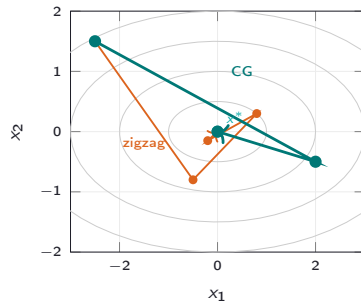
O mínimo ocorre em $x^* = A^{-1}b$.

Gradiente Descendente Simples

O gradiente de f é $\nabla f = Ax - b = -r$ (negativo do resíduo).

Ideia: mover na direção do resíduo $r^{(k)} = b - Ax^{(k)}$.

Problema: gradiente descendente “zigzagueia” e converge lentamente!



Algoritmo (para A SPD)

0. $x^{(0)}$ inicial; $r^{(0)} = b - Ax^{(0)}$; $p^{(0)} = r^{(0)}$

k. Para $k = 0, 1, 2, \dots$:

$$\alpha_k = \frac{\langle r^{(k)}, r^{(k)} \rangle}{\langle p^{(k)}, Ap^{(k)} \rangle}$$

$$x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}$$

$$r^{(k+1)} = r^{(k)} - \alpha_k Ap^{(k)}$$

$$\beta_k = \frac{\langle r^{(k+1)}, r^{(k+1)} \rangle}{\langle r^{(k)}, r^{(k)} \rangle}$$

$$p^{(k+1)} = r^{(k+1)} + \beta_k p^{(k)}$$

Propriedades notáveis

- Converge em **no máximo n passos** (exato em aritmética exata)
- Direções $p^{(k)}$ são **A -conjugadas**: $\langle p^{(i)}, Ap^{(j)} \rangle = 0$ para $i \neq j$
- Custo por iteração: $\mathcal{O}(\text{nnz})$ — apenas um produto Av !
- Não precisa armazenar A — só calcular Av

Quando usar

A **SPD** e **esparsa**. Na prática, converge em $k \ll n$ iterações.

Theorem (Estimativa de Convergência)

Para o método do Gradiente Conjugado:

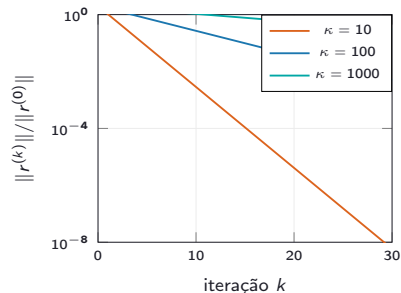
$$\frac{\|e^{(k)}\|_A}{\|e^{(0)}\|_A} \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k$$

em que $\kappa = \kappa(A)$ e o número de condição de A
e $\|v\|_A = \sqrt{v^T A v}$.

Impacto do condicionamento

κ grande $\Rightarrow$ muitas iterações.

Solução: **pré-condicionar** A !



Pré-condicionamento reduz κ efetivo $\Rightarrow$ menos iterações!

Gradiente Conjugado — Passo a Passo (Etapa 0)

Sistema SPD 2×2 — ideal para visualizar geometricamente

$$A = \begin{pmatrix} 4 & 1 \\ 1 & 3 \end{pmatrix}, \quad b = \begin{pmatrix} 9 \\ 7 \end{pmatrix}, \quad x^{(0)} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

Solução exata: $x^* = (2; 1,667)^T$

$$[4(2) + 1(5/3) = 9; \quad 1(2) + 3(5/3) = 7 \quad \checkmark]$$

Inicialização (antes da iteração 1):

$$r^{(0)} = b - Ax^{(0)} = \begin{pmatrix} 9 \\ 7 \end{pmatrix} - \begin{pmatrix} 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 9 \\ 7 \end{pmatrix}$$

$$p^{(0)} = r^{(0)} = \begin{pmatrix} 9 \\ 7 \end{pmatrix}$$

A primeira direção de busca é o **próprio resíduo**.

Verificar que A é SPD

$$\det(A) = 12 - 1 = 11 > 0 \quad \checkmark$$

$$a_{11} = 4 > 0 \quad \checkmark$$

$\Rightarrow$ CG é aplicável!

Algoritmo usa

- $r^{(k)}$: resíduo atual.
- $p^{(k)}$: direção de busca.
- α_k : tamanho do passo.
- β_k : atualização da direção.

Gradiente Conjugado — iteração $k = 1$

Estado atual: $x^{(0)} = (0; 0)^T$, $r^{(0)} = (9; 7)^T$, $p^{(0)} = (9; 7)^T$

Passo 1 — Calcular α_0 :

$$Ap^{(0)} = \begin{pmatrix} 4 & 1 \\ 1 & 3 \end{pmatrix} \begin{pmatrix} 9 \\ 7 \end{pmatrix} = \begin{pmatrix} 43 \\ 30 \end{pmatrix}$$

$$\alpha_0 = \frac{\langle r^{(0)}, r^{(0)} \rangle}{\langle p^{(0)}, Ap^{(0)} \rangle} = \frac{9^2 + 7^2}{9 \cdot 43 + 7 \cdot 30} = \frac{130}{597} \approx 0,218$$

Passo 2 — Atualizar x :

$$x^{(1)} = x^{(0)} + \alpha_0 p^{(0)} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} + 0,218 \begin{pmatrix} 9 \\ 7 \end{pmatrix} = \begin{pmatrix} 1,962 \\ 1,526 \end{pmatrix}$$

Passo 3 — Atualizar resíduo:

$$r^{(1)} = r^{(0)} - \alpha_0 Ap^{(0)} = \begin{pmatrix} 9 \\ 7 \end{pmatrix} - 0,218 \begin{pmatrix} 43 \\ 30 \end{pmatrix} = \begin{pmatrix} -0,374 \\ 0,460 \end{pmatrix}$$

Resultado de $k = 1$

$$x^{(1)} \approx \begin{pmatrix} 1,962 \\ 1,526 \end{pmatrix}$$

Erro: $\|x^{(1)} - x^*\| \approx 0,44$
(de $\|x^{(0)} - x^*\| \approx 2,49$)

Redução do erro

Em uma única iteração, o erro caiu $\approx 80\%$!

Gradiente Conjugado — iteração $k = 2$ (Convergência Exata!)

Estado atual: $r^{(1)} = (-0,374; 0,460)^T$

Passo 4 — Calcular β_0 e nova direção $p^{(1)}$:

$$\beta_0 = \frac{\langle r^{(1)}, r^{(1)} \rangle}{\langle r^{(0)}, r^{(0)} \rangle} = \frac{0,374^2 + 0,460^2}{130} = \frac{0,352}{130} \approx 0,00271$$

$$p^{(1)} = r^{(1)} + \beta_0 p^{(0)} = \begin{pmatrix} -0,374 \\ 0,460 \end{pmatrix} + 0,00271 \begin{pmatrix} 9 \\ 7 \end{pmatrix} = \begin{pmatrix} -0,349 \\ 0,479 \end{pmatrix}$$

Gradiente Conjugado — iteração $k = 2$ (Convergência Exata!)

Passo 5 — Calcular α_1 e $x^{(2)}$:

$$Ap^{(1)} = \begin{pmatrix} 4 \cdot (-0,349) + 0,479 \\ -0,349 + 3 \cdot 0,479 \end{pmatrix} = \begin{pmatrix} -0,917 \\ 1,088 \end{pmatrix}$$
$$\alpha_1 = \frac{0,352}{(-0,349)(-0,917) + (0,479)(1,088)} = \frac{0,352}{0,841} \approx 0,418$$
$$x^{(2)} = x^{(1)} + \alpha_1 p^{(1)} \approx \begin{pmatrix} 1,962 \\ 1,526 \end{pmatrix} + 0,418 \begin{pmatrix} -0,349 \\ 0,479 \end{pmatrix} = \begin{pmatrix} 1,816 \\ 1,726 \end{pmatrix}$$

Resultado final

$$x^{(2)} \approx (1,816; 1,726)^T$$

$$\text{Solução exata: } (2; \frac{5}{3})^T \approx (2; 1,667)^T$$

Erro residual: $\approx 0,2$ (erros de arredondamento no cálculo manual)

Teorema confirmado!

Para $A 2 \times 2$ SPD, CG converge em **no máximo 2 iterações** — exatamente o que ocorreu!

Prática

Para sistemas grandes, CG converge em $k \ll n$ iterações graças à estrutura das direções

Pré-Condicionamento — A Ideia

Problema com sistema mal-condicionado

Se $\kappa(A) \gg 1$, métodos iterativos convergem **muito lentamente**.

Solução: transformar o sistema em um equivalente com **melhor condicionamento**!

Pré-condicionamento à esquerda

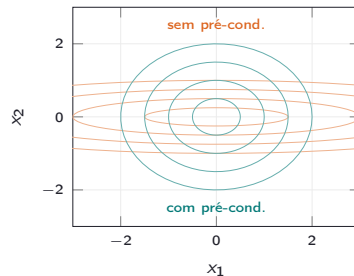
Dado pré-condicionador $M \approx A$ (fácil de inverter):

$$Ax = b \longrightarrow \underbrace{M^{-1}A}_{\kappa \approx 1} x = M^{-1}b$$

Se $M = A$: solução imediata (mas cara)

Se $M = I$: sem mudança (mas barata)

M ideal: entre os dois extremos.



Pré-condicionamento arredonda os contornos da função objetivo.

1. Jacobi (diagonal)

$$M = D = \text{diag}(a_{11}, \dots, a_{nn}).$$

- Custo: $\mathcal{O}(n)$ — trivial!
- Eficaz quando diagonal domina.
- Base para pré-cond. mais sofisticados.

2. Gauss-Seidel / SSOR

$$M = (D - L)D^{-1}(D - U)$$

- Melhor que Jacobi para muitas matrizes.
- SSOR: SOR simétrico — compatível com CG.

3. ILU — Fatoração LU Incompleta

Fatore $A \approx \tilde{L}\tilde{U}$ mantendo apenas os elementos no mesmo padrão de esparsidade de A :

$$M = \tilde{L}\tilde{U}$$

- **Muito eficaz** na prática.
- ILU(0): zeros em que A tem zeros.
- ILU(p): permite p níveis de *fill-in*.

4. Multigrid

Pré-condicionador **ótimo** para EDPs:
 $\kappa = \mathcal{O}(1)$, custo $\mathcal{O}(n)$. Tópico avançado!

Gradiente Conjugado Pré-Condicionado (PCG)

Algoritmo PCG

0. $x^{(0)}$; $r^{(0)} = b - Ax^{(0)}$; resolva $Mz^{(0)} = r^{(0)}$; $p^{(0)} = z^{(0)}$

k. Para $k = 0, 1, 2, \dots$:

$$\alpha_k = \frac{\langle r^{(k)}, z^{(k)} \rangle}{\langle p^{(k)}, Ap^{(k)} \rangle}$$

$$x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}$$

$$r^{(k+1)} = r^{(k)} - \alpha_k Ap^{(k)}$$

$$\text{resolva } Mz^{(k+1)} = r^{(k+1)}$$

$$\beta_k = \frac{\langle r^{(k+1)}, z^{(k+1)} \rangle}{\langle r^{(k)}, z^{(k)} \rangle}$$

$$p^{(k+1)} = z^{(k+1)} + \beta_k p^{(k)}$$

Custo por iteração

- 1 produto Ap ($\mathcal{O}(\text{nnz})$).
- 1 resolução $Mz = r$ (depende de M).
- Alguns produtos internos.

Por que funciona?

PCG converge em função de $\kappa(M^{-1}A)$ em vez de $\kappa(A)$.

Bom $M \Rightarrow \kappa(M^{-1}A) \approx 1 \Rightarrow$ pouquíssimas iterações!

Na prática (Python)

```
scipy.sparse.linalg.cg(A, b, M=M)
```

Fluxograma de Escolha do Método Iterativo

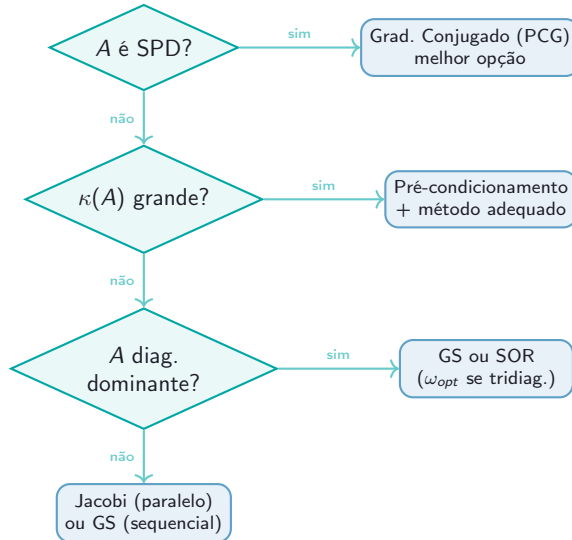


Tabela Comparativa dos Métodos Iterativos

Método	Custo/iter.	Paralel.	Garante conv.	Quando usar
Jacobi	$\mathcal{O}(n^2)$	✓	Diag. dom.	Paralelo, simples
Gauss-Seidel	$\mathcal{O}(n^2)$	×	Diag. dom., SPD	Sequencial, geral
SOR	$\mathcal{O}(n^2)$	×	$0 < \omega < 2$, SPD	EDPs, tridiagonais
Grad. Conjugado	$\mathcal{O}(\text{nnz})$	parcial	SPD	Matrizes SPD esparsas
PCG	$\mathcal{O}(\text{nnz})$	parcial	SPD	Recomendado para SPD

Recomendação prática

Use `scipy.sparse.linalg.cg` (CG) ou `gmres` (não-SPD) com pré-condicionador ILU via `spilu`.

Regra de ouro

- $n < 10^4$: método direto (LU)
- $n > 10^4$ e SPD: PCG + ILU
- $n > 10^4$ e não-SPD: GMRES + ILU

Gauss-Jacobi e Gauss-Seidel em Python

Gauss-Jacobi vetorizado

```
1 import numpy as np
2
3 def jacobi(A, b, tol=1e-8, maxiter=500):
4     n = len(b)
5     D_inv = 1.0 / np.diag(A)
6     R = A - np.diag(np.diag(A))
7     x = np.zeros(n)
8     for k in range(maxiter):
9         x_new = D_inv * (b - R @ x)
10        if np.linalg.norm(x_new - x,
11                           np.inf) < tol:
12            return x_new, k+1
13    x = x_new
14    return x, maxiter
```

Gauss-Seidel e SOR

```
1 def gauss_seidel(A, b, omega=1.0,
2                  tol=1e-8, maxiter=500):
3     n = len(b)
4     x = np.zeros(n)
5     for k in range(maxiter):
6         x_old = x.copy()
7         for i in range(n):
8             s = (np.dot(A[i,:i], x[:i])
9                  + np.dot(A[i,i+1:],
10                           x_old[i+1:]))
11             x_gs = (b[i] - s) / A[i,i]
12             x[i] = ((1-omega)*x_old[i]
13                    + omega*x_gs)
14             if np.linalg.norm(x - x_old,
15                               np.inf) < tol:
16                 return x, k+1
17     return x, maxiter
18 # omega=1.0 -> Gauss-Seidel
19 # omega>1.0 -> SOR
```

Uso

```
x_j, it_j = jacobi(A, b)
x_gs, it_gs = gauss_seidel(A, b)
x_sor, it_s = gauss_seidel(A, b, omega=1.5)
```

Custo por iteração

Jacobi: $\mathcal{O}(n^2)$
sequencial

GS/SOR: $\mathcal{O}(n^2)$

Gradiente Conjugado Pré-Condicionado em Python

PCG com SciPy (recomendado)

```
1 import numpy as np
2 import scipy.sparse as sp
3 import scipy.sparse.linalg as spla
4
5 # Exemplo: Laplaciano 2D
6 n = 50          # grade n x n
7 N = n*n         # sistema N x N
8
9 # Montagem da matriz esparsa
10 diags = [-1, 0, 1, -n, n]
11 vals = [-1, 4, -1, -1, -1]
12 A = sp.diags(vals, diags,
13             shape=(N, N)).toarray()
14 A = sp.csr_matrix(A)
15
16 b = np.ones(N)
17
18 # Pré-condicionador ILU
19 ilu = spla.spilu(A)
20 M = spla.LinearOperator(A.shape,
21                         lambda x: ilu.solve(x))
22
23 # Resolver com PCG
24 x, info = spla.cg(A, b, M=M, tol=1e-10)
25 print(f"Resíduo: {np.linalg.norm(A@x-b):.2e}")
26 print(f"Status: {info}") # 0 = convergiu
```

Comparação de métodos

```
1 # Comparação: Jacobi vs GS vs PCG
2 A_d = A.toarray()
3 b_d = b.copy()
4
5 # Jacobi
6 x_j, it_j = jacobi(A_d, b_d, 1e-6)
7
8 # Gauss-Seidel
9 x_gs, it_gs = gauss_seidel(A_d, b_d,
10                           tol=1e-6)
11 # SOR ótimo (formula de Young)
12 rho_j = 0.95 # calcular numericamente
13 w_opt = 2/(1+np.sqrt(1-rho_j**2))
14 x_s, it_s = gauss_seidel(A_d, b_d,
15                          omega=w_opt, tol=1e-6)
16
17 # PCG
18 x_cg, _ = spla.cg(A, b, M=M, tol=1e-6)
19
20 print(f"Jacobi:          {it_j} iters")
21 print(f"Gauss-Seidel: {it_gs} iters")
22 print(f"SOR (opt):        {it_s} iters")
```

Ex. 1 — Jacobi Manual

Resolva pelo método de Gauss-Jacobi com $x^{(0)} = (0,7; -1,6; 0,6)^T$ e $\varepsilon = 0,05$:

$$\begin{cases} 10x_1 + 2x_2 + x_3 = 7 \\ x_1 + 5x_2 + x_3 = -8 \\ 2x_1 + 3x_2 + 10x_3 = 6 \end{cases}$$

Verifique que a matriz é diagonal dominante antes de iterar.

Ex. 2 — Gauss-Seidel Manual

Resolva pelo método de Gauss-Seidel com $x^{(0)} = (0, 0, 0, 0)^T$ e $\varepsilon = 0,05$:

$$\begin{cases} x_1 + 0,5x_2 - 0,1x_3 + 0,1x_4 = 0,2 \\ 0,2x_1 + x_2 - 0,2x_3 - 0,1x_4 = -2,6 \\ -0,1x_1 - 0,2x_2 + x_3 + 0,2x_4 = 1,0 \\ 0,1x_1 + 0,3x_2 + 0,2x_3 + x_4 = -2,5 \end{cases}$$

Analise a convergência pelo critério de Sassenfeld.

Ex. 3 — SOR

Para o sistema tridiagonal 4×4 com $b_i = 2$ e diagonal principal 4, diagonais laterais -1 :

- (a) Calcule $\rho(T_J)$ numericamente
- (b) Calcule ω_{opt} pela fórmula de Young
- (c) Compare iterações de GS e SOR para $\varepsilon = 10^{-6}$

Fundamentos

- Iteração: $x^{(k+1)} = Cx^{(k)} + g$.
- Converge $\Leftrightarrow \rho(C) < 1$.
- Critério prático: diagonal dominância.
- Critério de parada relativo recomendado.

Jacobi e Gauss-Seidel

- Jacobi: paralelo, usa só $x^{(k)}$.
- GS: sequencial, usa $x^{(k+1)}$ imediatamente.
- GS $\approx 2\times$ mais rápido que Jacobi.
- Ambos: $\mathcal{O}(n^2)$ por iteração.

Métodos Avançados

- **SOR**: GS com parâmetro ω ; ω_{opt} pela fórmula de Young.
- **CG**: ótimo para SPD; converge em n passos.
- **PCG**: CG + pré-cond. reduz κ efetivo.
- Pré-cond.: Jacobi, ILU, Multigrid.

Regra de ouro

- $n < 10^4$: use método direto (LU).
- $n > 10^4$, SPD: `scipy.cg` + ILU.
- $n > 10^4$, geral: `scipy.gmres` + ILU.

Prática: `scipy.sparse.linalg.cg`, `gmres`, `spilu` — implementações industriais otimizadas.

- **Saad, Y.** (2003). *Iterative Methods for Sparse Linear Systems*. 2.^a ed. SIAM.
https://www-users.cse.umn.edu/~saad/IterMethBook_2ndEd.pdf — referência definitiva, gratuita.
- **Golub & Van Loan** (2013). *Matrix Computations*. 4.^a ed. JHU Press. Cap. 10.
- **Barrett et al.** (1994). *Templates for the Solution of Linear Systems*. SIAM.
<https://www.netlib.org/templates/> — guia prático gratuito.
- **Franco** (2007); **Ruggiero & Lopes** (1997) — referências brasileiras com exemplos.
- **Burden & Faires** (2015). *Numerical Analysis*. 10.^a ed. Cap. 7: métodos iterativos.
- **SciPy Sparse Linear Algebra**:
<https://docs.scipy.org/doc/scipy/reference/sparse.linalg.html>